ISP Final Exam - Paper 1

Course: INTRODUCTION TO SYSTEMS PROGRAMMING - A
Date: Thursday, July 9, 2026
Total Points: 100

Exercise 1 (5 points)

Consider the following recursive function:

```
static int mystery(int n) {
    if (n == 0) {
        return 1;
    } else {
        return n * mystery(n - 1);
    }
}
```

What does `mystery(4)` return?

a) It will return 1
b) It will return 4
c) It will return 24
d) It will return 120
e) It will produce a runtime error because of integer overflow
f) It will never return because the recursion doesn't terminate

---

Exercise 2 (6 points)

Consider the following recursive function:

```
static int f(int n) {
    if (n == 0) {
        return 0;
    } else {
        int digit = n % 10;
        n = n / 10;
        return digit + f(n);
    }
}
```

What does `f(12345)` return?

a) It will return 1
b) It will return 5
c) It will return 15
d) It will return 12345
e) It will produce a runtime error because the local variable `digit` is used after the recursive call
f) It will never return because `f(n)` calls `f(n/10)` and this enters an infinite loop

---

Exercise 3 (8 points)

Complete the following recursive function that calculates the sum of digits of a positive integer using recursion.

Precondition: `n >= 0`

```
static int sumOfDigits(int n) {
    // TODO: Complete this function
}
```

Exercise 4 (8 points)

Complete the following recursive function that generates all binary strings of length n. For example, `allBinaryStrings(2)` should return a list containing `["00", "01", "10", "11"]`.

Precondition: `n >= 0`

```
ArrayList<String> allBinaryStrings(int n) {
    ArrayList<String> result = new ArrayList<String>();
    if (n == 0) {
        result.add("");
    } else {
        // TODO: Complete this method
    }
    return result;
}
```

Exercise 5 (6 points)

Consider the following class representing a node in a doubly-linked list:

```
class Node {
    int element;
    Node next;
```

```
    Node prev;
}
```

Complete the following function `removeLast` that removes the last node from a linked list. The function receives a node which is part of a linked list (not necessarily the first node). You must first find the beginning of the list, then remove the last node.

```
static void removeLast(Node p) {
    // TODO: Complete this function
}
```

Exercise 6 (8 points)

Complete the `addAtBeginning` method of the linked list class. The method inserts a new node with element `x` at the beginning of the list.

```
class List {
    private Node first;

    // TODO: Complete this method
    void addAtBeginning(int x) {
        // TODO: Implementation
    }

    private class Node {
        int element;
        Node next;
    }
}
```

Exercise 7 (6 points)

Consider the following binary tree representation:

```
class TreeNode<T> {
    T key;
    TreeNode<T> leftChild;
    TreeNode<T> rightChild;
}
```

Complete the following function `countNodes` that returns the number of nodes in the binary tree.

```
static <T> int countNodes(TreeNode<T> root) {
    // TODO: Complete this function
}
```

Exercise 8 (8 points)

Complete the following function `mirrorTree` that receives a binary tree and modifies it so that it becomes its mirror image (left and right subtrees are swapped at every node).

```
static <T> void mirrorTree(TreeNode<T> root) {
    // TODO: Complete this function
}
```

Exercise 9 (6 points)

Consider the following classes:

```
class A {
    void print() {
        System.out.print("A");
    }
}

class B extends A {
    void print() {
        System.out.print("B");
    }
}

class C extends B {
    void print() {
        super.print();
        System.out.print("C");
    }
}
```

What will this code print when executed from the main method?

```
A obj1 = new C();
B obj2 = new B();
obj1.print();
obj2.print();
```

a) It will print AA
b) It will print ACB
c) It will print BC
d) It will print BAC
e) It will print CBB
f) It will not run because `obj1` is of type `A` which does not have a `print()` method

---

Exercise 10 (6 points)

Consider the following classes:

```
class Animal {
    void speak() {
        System.out.print("Animal");
    }
}

class Dog extends Animal {
    void speak() {
        System.out.print("Dog");
    }
}

class Cat extends Animal {
    void speak() {
        System.out.print("Cat");
    }
}
```

What will this code print?

```
ArrayList<Animal> animals = new ArrayList<Animal>();
animals.add(new Dog());
animals.add(new Cat());
animals.add(new Animal());

for (Animal a : animals) {
    a.speak();
}
```

a) It will print AnimalAnimalAnimal
b) It will print DogCatAnimal
c) It will print DogCatDog
d) It will print AnimalDogCat
e) It will produce a runtime error
f) It will not compile

Exercise 11 (8 points)

Define a class `LoggingCounter` that extends `Counter`. The `increment()` method should behave exactly as in `Counter` but additionally print a message "Counter incremented" each time it is called.

```
class Counter {
    protected int value;

    Counter() {
        this.value = 0;
    }

    void increment() {
        this.value++;
    }

    int getValue() {
        return this.value;
    }
}

// TODO: Define LoggingCounter class
```

Exercise 12 (6 points)

Consider the following interface:

```
interface Filter<T> {
    boolean accept(T element);
}
```

Define a class `PositiveFilter` that implements `Filter<Integer>` and accepts only positive integers.

```
// TODO: Define PositiveFilter class
```

Exercise 13 (8 points)

Complete the following function `filter` that receives a `Filter` and an `ArrayList`, and returns a new `ArrayList` containing only the elements that are accepted by the filter.

```
static <T> ArrayList<T> filter(Filter<T> filter, ArrayList<T> list) {
    // TODO: Complete this function
```

```
}
```

Exercise 14 (6 points)

Define a class `RangeIterator` that implements `Iterator<Integer>` and generates all integers from `start` (inclusive) to `end` (exclusive).

```
// TODO: Define RangeIterator class
```

Exercise 15 (5 points)

What is the purpose of buffering in input/output operations?

a) To compress data before writing to disk
b) To reduce the number of system calls by reading/writing data in chunks
c) To encrypt data for security
d) To convert data between different encodings
e) To automatically close files when the program ends
f) To make files larger for better performance

Exercise 16 (6 points)

Consider the following code for reading a file:

```
try {
    FileInputStream fis = new FileInputStream("data.txt");
    int b = fis.read();
    fis.close();
} catch (IOException e) {
    System.out.println("Error reading file");
}
```

What is the problem with this code?

a) The file will not be closed if an exception occurs
b) It should use `FileReader` instead of `FileInputStream`
c) The `read()` method should be called in a loop
d) The `catch` block should not print a message
e) The file name must be enclosed in double quotes
f) There is no problem; the code is correct